GRC102 Security Governance Simulation Assessment Brief

Practical Lab 8 Monitoring Auditing Controls and Executive Reporting

Course	GRC102 Information Security Governance
Assessment	Individual practical laboratory 100 marks
Submission	GitHub repository and associated evidence

1. Docker Network Configuration

An isolated Docker network named secgov_network was created for the security governance simulation. The network uses the Docker bridge driver and provides the communication layer for the simulated web server, database server and monitoring server.

The network was verified using:

sudo docker network ls

The output confirmed that secgov_network was available with a local bridge configuration.

Figure 2: Docker secgov_network configuration



Docker Compose Configuration Validation

The Docker Compose configuration was validated before starting the simulated environment.

The configuration defines three containers:

- **Web server:** web_server, using the vulhub/struts2:s2-045 image and exposing port 8080.
- **Database server:** database_server, using MySQL 5.7 with the customer_data database.
- **Monitoring server:** monitoring_server, using Ubuntu 20.04 and the mounted monitoring directory.

The services are configured to use the isolated secgov_network.

The Compose validation completed successfully. A warning indicated that the version attribute is obsolete and would be ignored by the installed Compose implementation; this did not prevent the configuration from being processed.

Docker Compose configuration validation: Evidence 03

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ sudo docker-compose config
[sudo] password for icdfa:
WARN[0000] /home/icdfa/security_governance_lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, plea
name: security_governance_lab
services:
  database:
    container_name: database_server
    environment:
      MYSQL_DATABASE: customer_data
      MYSQL_PASSWORD: app_password
      MYSQL_ROOT_PASSWORD: password
      MYSQL_USER: app_user
    image: mysql:5.7
    networks:
      secgov_network: null
    volumes:
      - type: bind
        source: /home/icdfa/security_governance_lab/database_init
        target: /docker-entrypoint-initdb.d
        bind:
          create_host_path: true
  monitoring:
    command:
      - tail
      - -f
      - /dev/null
    container_name: monitoring_server
    image: ubuntu:20.04
    networks:
      secgov_network: null
    volumes:
      - type: bind
        source: /home/icdfa/security_governance_lab/monitoring
        target: /monitoring
        bind:
          create_host_path: true
  web:
    container_name: web_server
    image: vulhub/struts2:s2-045
    networks:
      secgov_network: null
    ports:
      - mode: ingress
        target: 8080
        published: "8080"
        protocol: tcp
networks:
```

Running Simulated Environment

The Docker-based security governance simulation was successfully started within the isolated laboratory environment.

Three containers were deployed:

- **Web server:** web_server
- **Database server:** database_server
- **Monitoring server:** monitoring_server

The services were connected to the isolated secgov_network. The web application was exposed on local port 8080 for authorised laboratory testing.

Figure 4: Running Docker containers

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ sudo docker-compose ps
WARN[0000] /home/icdfa/security_governance_lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                  COMMAND                  SERVICE      CREATED          STATUS          PORTS
database_server     mysql:5.7              "docker-entrypoint.s…"   database     50 seconds ago   Up 48 seconds   3306/tcp, 33060/tcp
monitoring_server   ubuntu:20.04           "tail -f /dev/null"      monitoring   50 seconds ago   Up 48 seconds
web_server          vulhub/struts2:2.3.30  "/usr/local/bin/mvn-…"   web          50 seconds ago   Up 48 seconds   0.0.0.0:8080→8080/tcp, [::]:8080→8080/tcp

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

Note: The original laboratory image tag vulhub/struts2:s2-045 was unavailable in the current Docker registry. The web service was therefore configured with the available vulhub/struts2:2.3.30 image while retaining the intended S2-045/CVE-2017-5638 simulation objective.

Vulnerability Scan Results

The vulnerability scanner was executed against the isolated Docker-based security governance simulation on 8 October 2026. The scan tested three conditions: Apache Struts S2-045 (CVE-2017-5638), MySQL default credentials, and network segmentation between the web and database services.

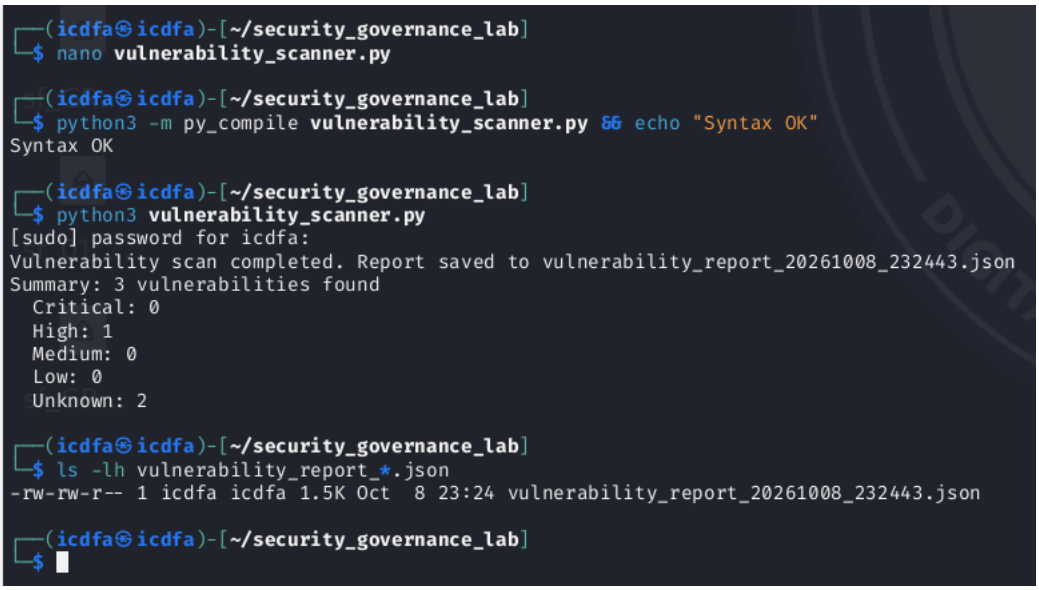
The initial scan generated the file vulnerability_report_20261008_232443.json. The results contained one confirmed high-severity finding and two inconclusive checks caused by technical errors in the scanner.

Security Check	Observed Result	Assessment
Apache Struts S2-045	The scanner could not resolve the hostname web_server when connecting from the host environment.	Inconclusive; vulnerability status not established.
MySQL default credentials	MySQL 5.7.44 was accessible using the default root credentials configured for the laboratory.	High-severity finding confirmed by the scanner.
Network segmentation	The check encountered a UTF-8 decoding error while processing the response from the database service.	Inconclusive; segmentation status not established.

The scanner summary recorded three checks, with one high-severity result and two results classified as unknown. The total of three therefore represents the checks performed, not three confirmed vulnerabilities.

The confirmed MySQL finding indicates that default database credentials remain a security weakness in the simulated environment. The two inconclusive results indicate that the scanner requires technical correction before its findings can reliably establish the Struts vulnerability status and network reachability.

Figure 5: Initial Vulnerability Scan Results



Evidence file: vulnerability_report_20261008_232443.json

Database Network Reachability Test

A connectivity test was performed from the simulated web container to the database container on TCP port 3306.

The test command was:

sudo docker exec web_server bash -c 'timeout 3 bash -c "</dev/tcp/database_server/3306"'

The command returned exit code 0, indicating that the TCP connection succeeded. This demonstrates that the web container could reach the database service through the laboratory network.

From a governance perspective, this reachability is relevant because unrestricted connectivity between application and database tiers can increase the potential for lateral movement if the web application is compromised. The result indicates a segmentation concern that should be assessed and addressed through least-privilege network controls.

Figure 6: Baseline Web-to-Database Connectivity Test



Evidence filename: 06_Web_DB_Connectivity_Baseline.png

Vulnerability Assessment Baseline

The vulnerability scanner was executed against the isolated Docker-based security governance simulation on 9 October 2026. The assessment evaluated three security conditions: Apache Struts S2-045 (CVE-2017-5638), MySQL default credentials, and network segmentation between the web and database services.

The scan completed and generated the JSON report vulnerability_report_20261009_000344.json. The scanner summary reported one critical result, one high-severity result and one result with unknown severity.

Severity	Number of Results
Critical	1
High	1
Medium	0
Low	0
Unknown	1
Total checks	**3**

These results establish the initial vulnerability-assessment baseline. The critical and high-severity results require examination of their individual findings and remediation recommendations. The unknown result requires further validation before its security status can be determined.

Initial Vulnerability Scan Summary



Evidence file: vulnerability_report_20261009_000344.json

Governance Tracker Initialisation

A Python-based security governance tracker was initialised using the laboratory's sample data. The tracker records security policies, controls, risks, incidents, metrics and audits in a JSON data file.

The initialisation process created three policies, three controls, three risks, one simulated incident, three metrics and one audit record. These records provide a baseline for demonstrating how governance information can be documented and summarised for oversight.

The tracker generates a dashboard summary that allows governance stakeholders to review the number of registered records and the status of selected security metrics.

Figure 7: Security Governance Tracker Dashboard

```
python3 governance_tracker.py --dashboard
{
  "generated_date": "2026-10-09T00:25:11.958566",
  "summary": {
    "policies": 3,
    "controls": 3,
    "risks": 3,
    "incidents": 1,
    "metrics": 3,
    "audits": 1
  },
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": [
    {
      "name": "Patch Compliance",
      "target": 95.0,
      "actual": 85.0,
      "status": "Red"
    },
    {
      "name": "Vulnerability Remediation Time",
      "target": 7.0,
      "actual": 12.0,
      "status": "Green"
    },
    {
      "name": "Security Incidents",
      "target": 0.0,
      "actual": 1.0,
      "status": "Green"
    }
  ]
}

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ ls -lh governance_data.json
-rw-rw-r-- 1 icdfa icdfa 4.6K Oct  9 00:21 governance_data.json

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

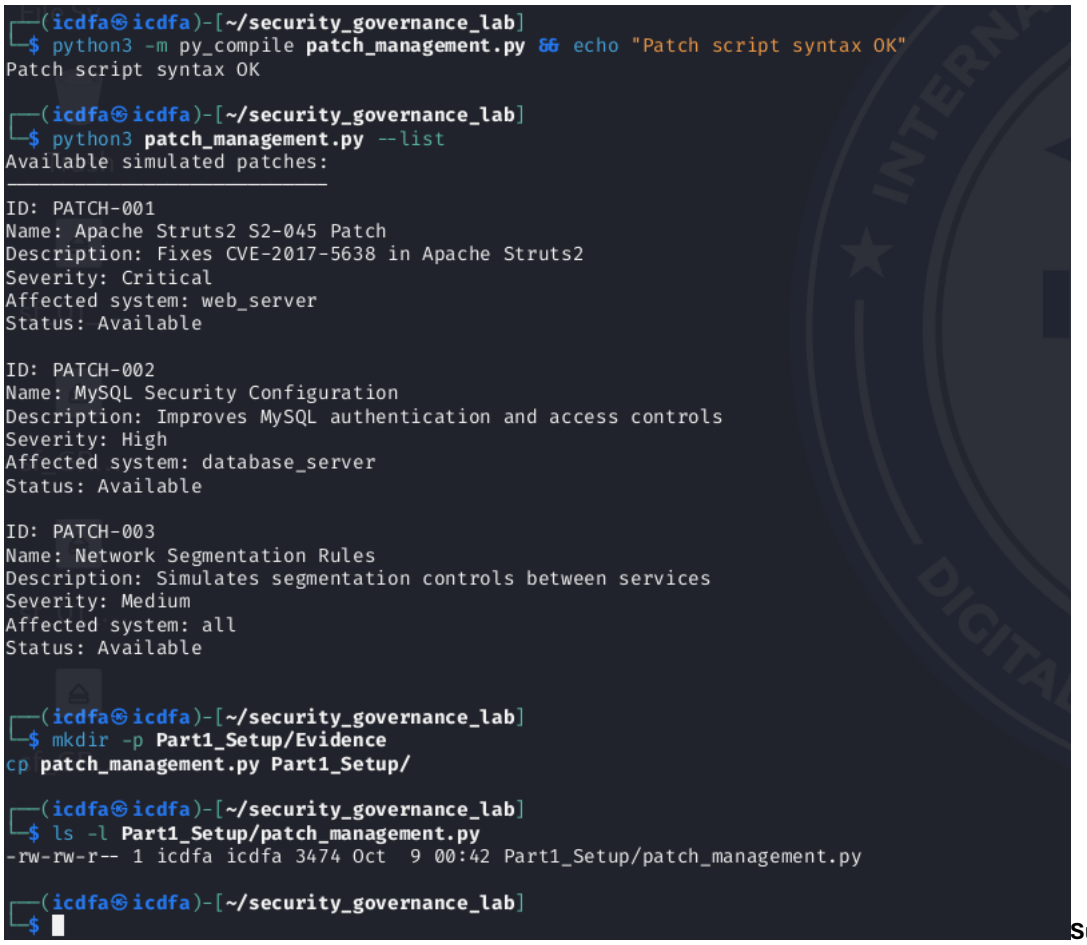
Supporting evidence: governance_tracker.py, initialize_governance.py and governance_data.json.

Patch-Management System Setup

A Python-based patch-management simulation was created to catalogue remediation actions for the security weaknesses identified in the laboratory. The script defines three remediation items: a critical Apache Struts S2-045 patch, a high-severity MySQL security configuration, and medium-severity network-segmentation rules.

The script was configured to list the available remediation items and record simulated patch status for subsequent governance analysis. At this stage, the remediation items were catalogued but not applied.

Figure 8: Available Simulated Patch-Management Actions



Supporting evidence: patch_management.py

Implementation limitation: Recording a simulated patch status does not itself update the vulnerable application, change database credentials, or enforce network restrictions. These controls must be assessed separately during Part 3

Part 2 – Analysing Security Governance Failures

2.1 Attack Simulation

An authorised simulation was performed against the locally hosted Struts web application within the isolated Docker laboratory. The script sent a request to the application and checked whether the response contained the expected vulnerability marker associated with the Struts S2-045 simulation.

The request returned HTTP status 200, and the vulnerability marker was observed. The script recorded the result in attack_log.json, with the success indicator set to true. The log also recorded three simulated records as affected.

The activity was restricted to the laboratory simulation. No real customer data was accessed or exfiltrated.

Figure 9: Struts Attack Simulation Output

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ python3 simulate_attack.py
Testing the isolated Struts lab application ...
The vulnerability marker was returned.
Recording the simulated impact; no real data is accessed.
Simulation result saved to attack_log.json

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat attack_log.json
{
  "timestamp": "2026-10-09T00:48:45.897912",
  "attack_type": "Struts S2-045 lab simulation",
  "target": "local web_server on port 8080",
  "success": true,
  "simulation_only": true,
  "result": "Vulnerability marker observed",
  "http_status": 200,
  "data_accessed": "Simulated customer_data database",
  "records_affected": 3
}

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

Evidence file: attack_log.json

2.2 Initial Interpretation

The observed vulnerability marker indicates that the application's response matched the condition checked by the simulation script. When considered alongside the critical Struts finding in the baseline vulnerability report, this result supports further investigation of the simulated web application's exposure to CVE-2017-5638.

The simulation does not independently demonstrate that a full remote-code-execution attack occurred or that real customer information was obtained. The finding should therefore be interpreted as laboratory evidence of the simulated vulnerability condition, not as proof of real data compromise.

2.3 Governance Failure Analysis

The governance failure analysis script examined the vulnerability assessment report, attack simulation log, governance tracker data and recorded patch status. The analysis generated a JSON report identifying eight governance issues across seven categories.

Governance Failure	Number of Findings	Governance Significance
Patch Management	2	Critical and high-severity vulnerabilities require remediation and verification.
Incident Detection	1	The simulation returned the expected vulnerability marker, highlighting the need to validate security monitoring and alerting.
Policy Implementation	1	The existence of a patch-management policy does not establish that the policy is operating effectively.
Vulnerability Management	1	The critical Struts finding requires prioritised remediation and independent retesting.

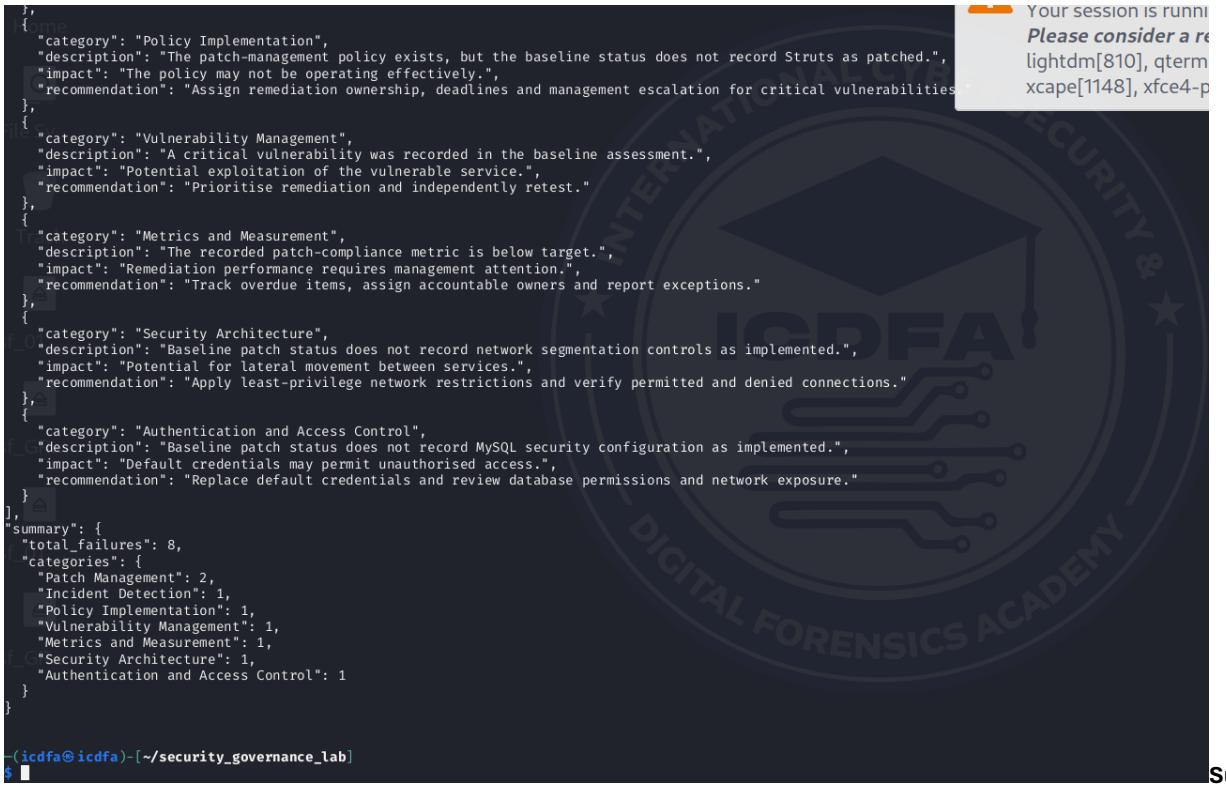
Metrics and Measurement	1	The patch-compliance metric was below its recorded target and requires management attention.
Security Architecture	1	The baseline did not record network segmentation as implemented; separate testing confirmed web-to-database TCP connectivity.
Authentication and Access Control	1	MySQL was accessible using the laboratory's configured default root credentials.
Total	**8**	

2.3.1 Interpretation

The results demonstrate that technical weaknesses and governance deficiencies are connected. An unpatched vulnerable application and weak database authentication increase technical risk, while incomplete segmentation can allow greater access between application and data services. These conditions require accountable control owners, remediation deadlines, monitoring and verification.

The analysis also highlights the importance of effective governance oversight. Policies and metrics are useful only when they lead to action, exceptions are escalated appropriately, and remediation is independently verified. The findings are based on the simulated environment and should not be interpreted as evidence of a real-world breach.

Figure 10: Governance Failure Analysis Results



Supporting evidence: analyze_governance_failures.py, governance_failure_report_*.json, vulnerability_report_20261009_000344.json, attack_log.json and governance_data.json

Part 3 – Implementing Security Governance Controls

3.1 Purpose

The purpose of Part 3 was to implement and verify governance controls addressing the security weaknesses identified in the baseline assessment and governance failure analysis.

The control implementation focuses on patch management, database authentication and access control, network segmentation, security monitoring, and executive governance oversight.

The laboratory's patch-management script provides simulated remediation status for selected controls. These recorded statuses will be evaluated alongside independent technical checks so that the report distinguishes simulated control records from verified technical changes.

3.2 Remediation Priorities

The baseline findings were used to prioritise remediation. The critical Apache Struts S2-045 finding was assigned the highest priority, followed by the high-severity MySQL default-credentials finding and the network-segmentation concern.

Each control will be assessed through implementation evidence, validation results, documented ownership and follow-up actions. A control will not be considered technically effective solely because its simulated status has been marked as complete.

11_**Available_Remediation_Actions**

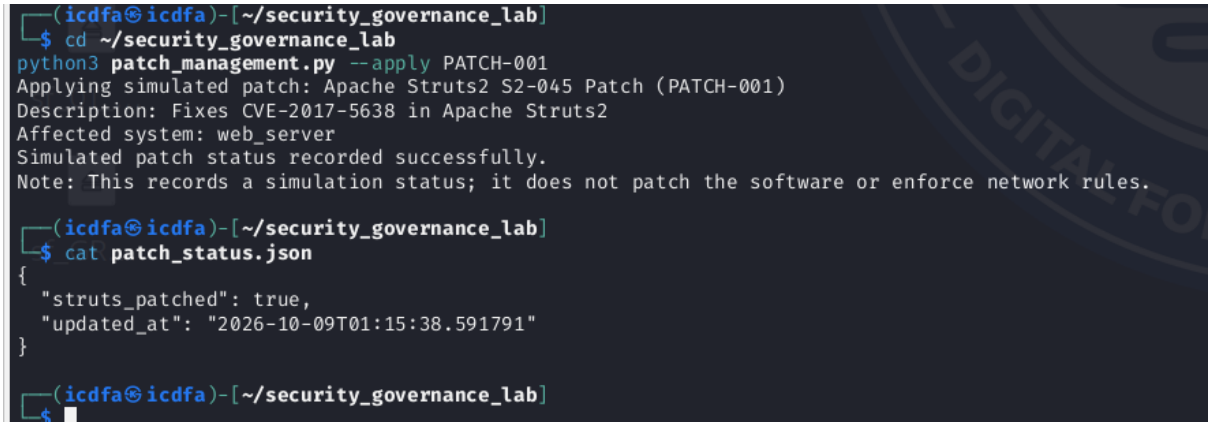


3.3 Patch Management — Apache Struts

The patch-management simulation was used to record the remediation status for the critical Apache Struts S2-045 vulnerability (CVE-2017-5638). The remediation action was identified as PATCH-001.

The script records the simulated patch status in patch_status.json for use by the governance analysis and control-verification processes.

Figure 12: Simulated Struts Patch Status



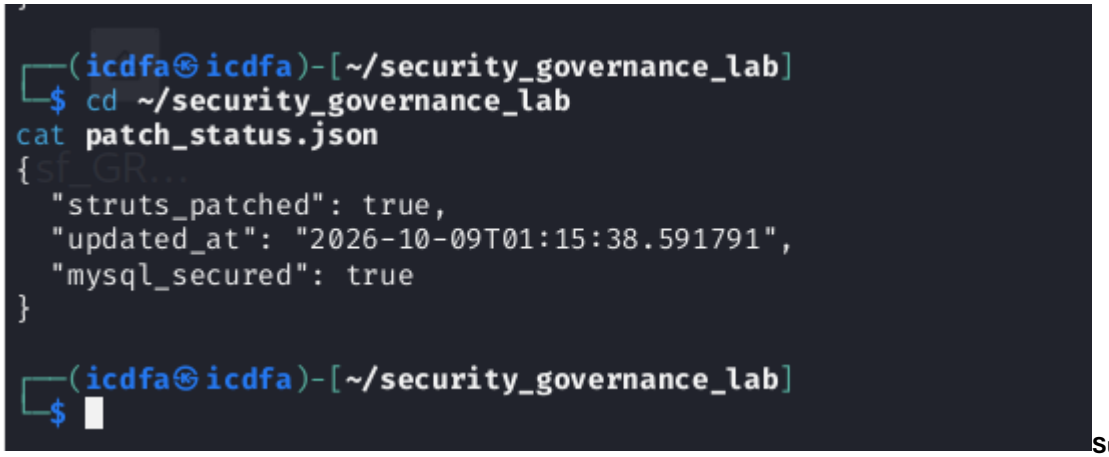
Verification limitation: Recording the patch status does not upgrade the Struts software or prove that the vulnerability has been removed. Technical verification will be required before the control can be reported as effective.

3.4 Simulated Remediation Status

The patch-management simulation recorded the remediation status of three control areas: Apache Struts vulnerability management, MySQL security configuration, and network segmentation. The resulting patch_status.json file recorded all three status flags as true.

These records demonstrate that the remediation workflow can track the simulated completion of identified actions. However, the status flags do not independently prove that the vulnerable application was patched, database credentials were changed, or network restrictions were enforced.

Figure 13: Recorded Remediation Status



Supporting evidence: patch_management.py and patch_status.json.

Outstanding assurance activity: Validate the live application, database authentication and network connectivity independently before concluding that the controls are technically effective.

3.4 Independent Technical Verification

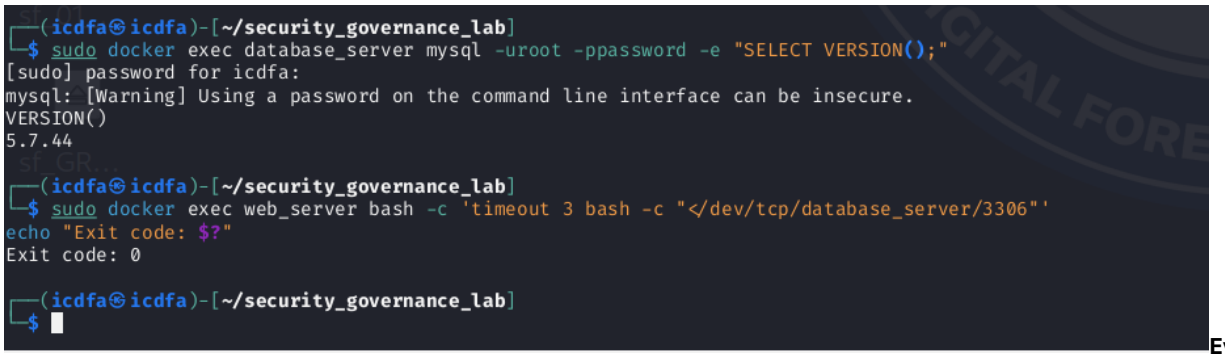
Independent technical checks were performed after the patch-management simulation recorded the Struts, MySQL and network-segmentation controls as complete.

The MySQL test successfully executed SELECT VERSION() using the laboratory's configured root credentials and returned version 5.7.44. The command therefore confirmed that the default root password remained accepted by the database.

A separate TCP connectivity test from the web container to database_server on port 3306 returned exit code 0, confirming that the web container could still reach the database service.

These results demonstrate a difference between the simulated remediation records and the observed technical state. Although the patch-status file recorded the controls as complete, the database authentication and network-segmentation weaknesses remained present during testing. The controls must not be reported as technically effective until corrective changes have been implemented and successfully retested.

Figure 14: Independent Database Authentication and Network Reachability Tests



Evidence: patch_status.json and the recorded terminal output.

3.5 Security Monitoring Controls

A local security-monitoring script was executed to collect Docker container status, search the web-container logs for configured suspicious patterns, and test connectivity between the web and database services.

The run completed and generated timestamped evidence files. No matching suspicious patterns were found in the collected web logs. However, the connectivity check generated an alert because the web container could reach the database on TCP port 3306.

The database-connectivity alert demonstrates how monitoring can highlight a control weakness requiring investigation and escalation. The absence of matching patterns in the collected logs does not establish that the environment is free from suspicious activity; it only describes the results of this particular collection and search.

Figure 15: Security Monitoring Output and Database Connectivity Alert

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ ls -lh monitoring/output/
total 8.0K
-rw-rw-r-- 1 icdfa icdfa 203 Oct  9 01:32 container_status_20261009_013240.txt
-rw-rw-r-- 1 icdfa icdfa 152 Oct  9 01:32 security_alerts_20261009_013240.log
-rw-rw-r-- 1 icdfa icdfa   0 Oct  9 01:32 suspicious_web_events_20261009_013240.log

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat "$(ls -t monitoring/output/security_alerts_*.log | head -n 1)"
Security monitoring run: 2026-10-09T01:32:40+01:00
No matching patterns found in collected web logs.
ALERT: Web container can reach database port 3306.

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

Supporting evidence: monitoring/security_monitor.sh and the timestamped files in monitoring/output/.

3.6 Governance Oversight and Accountability

A governance-oversight document was prepared to define proposed responsibilities for the CEO, CISO, CIO and CFO, together with reporting arrangements for security risks and unresolved control failures.

The document establishes that technical control owners are responsible for corrective actions, while security governance provides oversight, escalation and review of remediation evidence. It recommends that critical or overdue findings be reported to senior management and that closure decisions be supported by verification evidence.

The document represents a proposed governance arrangement for the laboratory simulation. Its creation does not demonstrate formal approval or implementation by a real organisation.

Figure 16: Security Governance Oversight Document

```
└─$ cat Part3_Control_Implementation/governance_oversight.md
# Security Governance Oversight

## 1. Purpose

This document defines proposed responsibilities and reporting arrangements for the security governance simulation.

## 2. Executive Responsibilities

- CEO: Overall accountability for security governance.
- CISO: Security programme oversight, risk escalation and monitoring of remediation.
- CIO: Oversight of IT infrastructure and technical control implementation.
- CFO: Oversight of the security budget and resources required for remediation.

## 3. Board Oversight

The Board should receive periodic security governance reports covering significant vulnerabilities, overdue remediation, security incidents, control effectiveness

Critical findings that exceed approved remediation deadlines should be escalated to senior management according to the organisation's risk and escalation procedur

## 4. Reporting and Accountability

Security and IT control owners are responsible for implementing assigned corrective actions and providing evidence of completion.

The CISO or designated assurance function should review high-risk exceptions, monitor remediation deadlines and arrange independent retesting where appropriate.

Management should document risk-acceptance decisions, accountable owners, agreed deadlines and compensating controls.

## 5. Review

Governance arrangements should be reviewed periodically to ensure that responsibilities remain clear and management reporting supports timely decisions and contir

## 6. Document Status

This is a proposed governance arrangement for the laboratory simulation. Its creation does not demonstrate formal approval or implementation by a real organisatio

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ find Part3_Control_Implementation/monitoring -maxdepth 2 -type f
Part3_Control_Implementation/monitoring/output/container_status_20261009_013240.txt
Part3_Control_Implementation/monitoring/output/suspicious_web_events_20261009_013240.log
Part3_Control_Implementation/monitoring/output/security_alerts_20261009_013240.log
Part3_Control_Implementation/monitoring/security_monitor.sh

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

Supporting evidence: governance_oversight.md.

3.7 Lab Network Exposure Restriction

The Docker Compose configuration was updated to bind the web application's published port to the Kali VM's loopback interface (127.0.0.1). The web container was restarted, and the effective port mapping was verified using sudo docker port web_server.

The command returned 8080/tcp -> 127.0.0.1:8080, confirming that the published web-service port is bound to localhost rather than all network interfaces. This configuration supports the requirement to keep the intentionally vulnerable application isolated while allowing authorised local laboratory testing.

Figure 17: Localhost Port-Binding Verification



Evidence file: 17_Localhost_Port_Verification.png

Verification result: Localhost binding confirmed. Other network-access paths and the overall VM network configuration should also be reviewed to ensure the application remains isolated.

3.8 Control Verification Results

The control-verification script was executed to review the recorded remediation statuses, the existence of the security-monitoring script and governance-oversight document, and the governance tracker records.

The script confirmed that the patch-status file recorded the Struts, MySQL and network-segmentation controls as true. It also confirmed that the monitoring script and governance-oversight document existed. The governance tracker contained four policies, eight controls, three risks and three metrics.

These results demonstrate that the control workflow and associated governance records were established. However, the recorded statuses do not independently demonstrate technical effectiveness. Earlier verification tests confirmed that MySQL still accepted the configured default root password and that the web container could reach the database on TCP port 3306.

The remaining assurance requirement is to repeat the vulnerability and connectivity tests, compare the results with the baseline, and document any control that remains ineffective.

Figure 18: Control Verification Summary

```
Important: recorded status flags indicate simulation state only. Confirm technical effectiveness through independent tests.

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ python3 implement_governance_controls.py --verify

=== Control Verification Summary ===
struts_patched recorded: True
mysql_secured recorded: True
network_segmented recorded: True
Monitoring script exists: True
Governance oversight document exists: True
Policies recorded: 4
Controls recorded: 8
Risks recorded: 3
Metrics recorded: 3

Important: recorded status flags indicate simulation state only. Confirm technical effectiveness through independent tests.

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
```

Supporting evidence: implement_governance_controls.py, patch_status.json, governance_data.json, and governance

3.9 Post-Control Retesting

Following the simulated control implementation, the Struts S2-045 laboratory check was executed again. The updated attack_log.json recorded success: true, an HTTP status of 200, and the result Vulnerability marker observed.

The retest therefore did not demonstrate that the vulnerability condition had been removed. Although the patch-management workflow recorded the Struts patch status as true, the application continued to return the marker checked by the simulation. This demonstrates the difference between recording remediation completion and verifying technical effectiveness.

The result requires further investigation and remediation of the application image, followed by another vulnerability assessment. The log identifies the activity as a simulation and does not establish that actual customer data was accessed or exfiltrated.

Figure 19: Post-Control Struts Retest Result

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat attack_log.json
{
  "timestamp": "2026-10-09T02:03:13.079598",
  "attack_type": "Struts S2-045 lab simulation",
  "target": "local web_server on port 8080",
  "success": true,
  "simulation_only": true,
  "result": "Vulnerability marker observed",
  "http_status": 200,
  "data_accessed": "Simulated customer_data database",
  "records_affected": 3
}

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

Evidence file: post_control_attack_log.json

Retest status: Failed — vulnerability marker still observed.

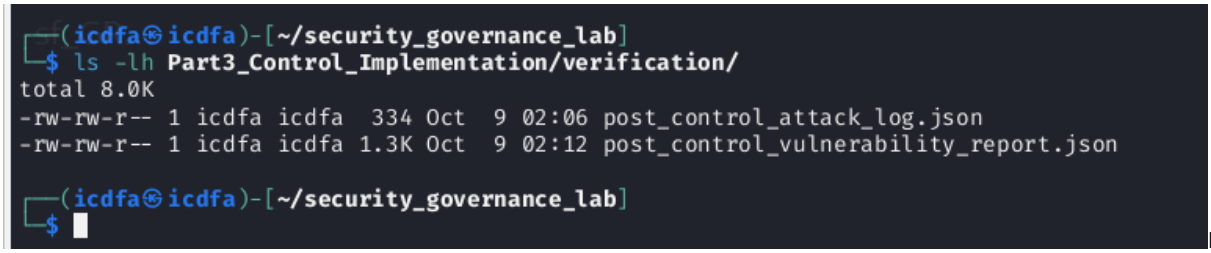
3.10 Post-Control Vulnerability Assessment

A second vulnerability scan was executed after the simulated control implementation. The scan completed and generated vulnerability_report_20261009_020958.json.

The post-control summary recorded one critical finding, one high-severity finding and one finding with unknown severity. These counts were unchanged from the baseline scan. The results therefore do not demonstrate a reduction in the reported severity counts following the simulated remediation.

This outcome reinforces the importance of technical verification. Recording a remediation status as complete is not sufficient evidence that a vulnerability has been removed. The individual findings must be reviewed, the underlying technical controls corrected where necessary, and the assessment repeated.

Figure 20: Post-Control Vulnerability Scan



Evidence file: post_control_vulnerability_report.json

Assessment status: Improvement in the vulnerability scan summary has not been demonstrated.

4.1 Governance Metrics and Reporting

A Python-based metrics script was created to generate five security-governance trend visualisations, an executive dashboard, a Board report and a JSON data file. The outputs demonstrate how patch compliance, vulnerability remediation time, security incidents, monitoring coverage and policy compliance can be presented for management review.

The dashboard separately reports the observed laboratory findings. The critical Struts finding and high-severity MySQL default-credentials finding remain unresolved or unverified, and web-to-database connectivity was confirmed. These findings are kept distinct from the illustrative trend data used in the charts.

The illustrative charts demonstrate a possible reporting format; they are not actual organisational performance measurements. Management decisions should be based on verified evidence, accountable remediation actions and independent retesting rather than assumed improvement.

Figure 21: Metrics Script Output

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ python3 Part4_Metrics_Reporting/governance_metrics.py
Security governance metrics and reports generated.
Executive dashboard: /home/icdfa/security_governance_lab/Part4_Metrics_Reporting/executive_dashboard.html
Board report: /home/icdfa/security_governance_lab/Part4_Metrics_Reporting/board_report.md
Metrics data: /home/icdfa/security_governance_lab/Part4_Metrics_Reporting/governance_metrics_data.json
Visualisations: 5 charts
Trend data is illustrative; lab findings reflect recorded tests.

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ ls -lh Part4_Metrics_Reporting/
total 36K
-rw-rw-r-- 1 icdfa icdfa 2.2K Oct  9 02:22 board_report.md
drwxrwxr-x 2 icdfa icdfa 4.0K Oct  9 02:19 Evidence
-rw-rw-r-- 1 icdfa icdfa 3.3K Oct  9 02:22 executive_dashboard.html
-rw-rw-r-- 1 icdfa icdfa 2.5K Oct  9 02:22 governance_metrics_data.json
-rw-rw-r-- 1 icdfa icdfa 8.7K Oct  9 02:21 governance_metrics.py
drwxrwxr-x 2 icdfa icdfa 4.0K Oct  9 02:22 metrics_visualizations
drwxrwxr-x 2 icdfa icdfa 4.0K Oct  9 02:21 __pycache__

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ ls -lh Part4_Metrics_Reporting/metrics_visualizations/
total 360K
-rw-rw-r-- 1 icdfa icdfa 68K Oct  9 02:22 monitoring_coverage.png
-rw-rw-r-- 1 icdfa icdfa 65K Oct  9 02:22 patch_compliance.png
-rw-rw-r-- 1 icdfa icdfa 74K Oct  9 02:22 policy_compliance.png
-rw-rw-r-- 1 icdfa icdfa 76K Oct  9 02:22 security_incidents.png
-rw-rw-r-- 1 icdfa icdfa 69K Oct  9 02:22 vulnerability_remediation.png

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

Supporting evidence: governance_metrics.py, governance_metrics_data.json, executive_dashboard.html, board_report.md and the generated trend charts.

4.2 Executive Dashboard and Visualisations

The security governance metrics script generated an HTML executive dashboard and five trend visualisations covering patch compliance, vulnerability remediation time, security incidents, monitoring coverage and policy compliance.

The dashboard also presents the verified laboratory findings, distinguishing unresolved technical weaknesses from illustrative training metrics. This separation helps management review the evidence without treating simulated values or recorded patch statuses as proof of control effectiveness.

The trend charts demonstrate a reporting format for analysing performance over time. Their illustrative values should not be interpreted as measurements of an actual organisation.

Figure 21: Security Governance Executive Dashboard

Security Governance Executive Dashboard

Generated: 2026-10-09T02:25:12.395965

Assurance status: Critical and high-severity findings remain unresolved or unverified. Simulated patch flags are not proof of technical remediation.

Verified Laboratory Findings

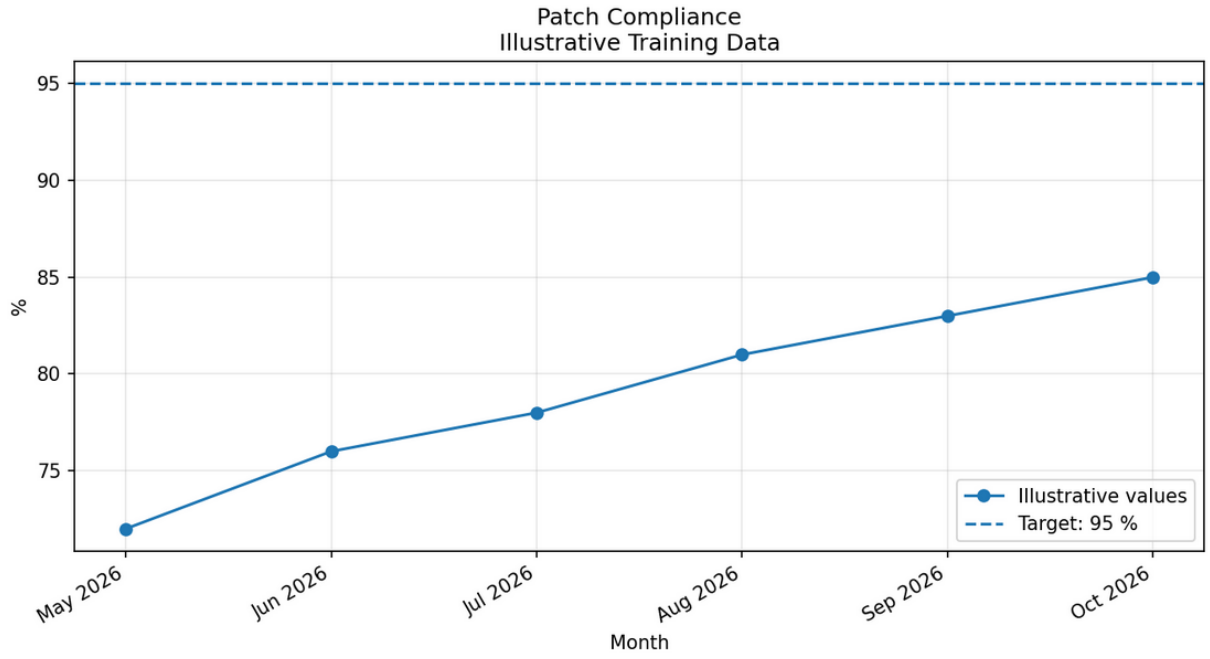
Issue	Severity	Status	Evidence
Apache Struts S2-045	Critical	Open - retest unsuccessful	The post-control scanner still reports the critical finding, and the lab check still returns its vulnerability marker.
MySQL default root credentials	High	Open - remediation not verified	The configured default root password was still accepted by the database during technical verification.
Web-to-database connectivity	Review required	Connectivity confirmed	The web container could reach database TCP port 3306.
Network segmentation assessment	Inconclusive	Further testing required	The scanner's network check encountered a UTF-8 decoding error.

Illustrative Metrics Trends

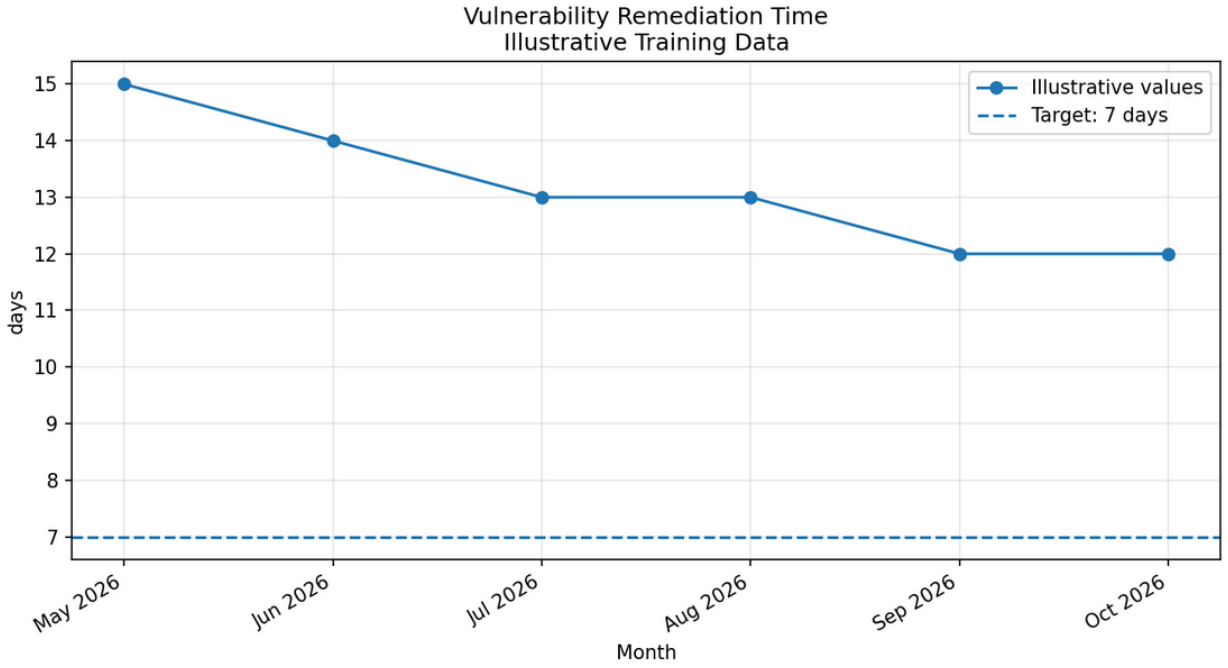
The charts below demonstrate a possible reporting format using illustrative training data. They do not represent measured company performance or prove that controls improved.

Figure 22: Security Governance Metrics Visualisations

Patch Compliance

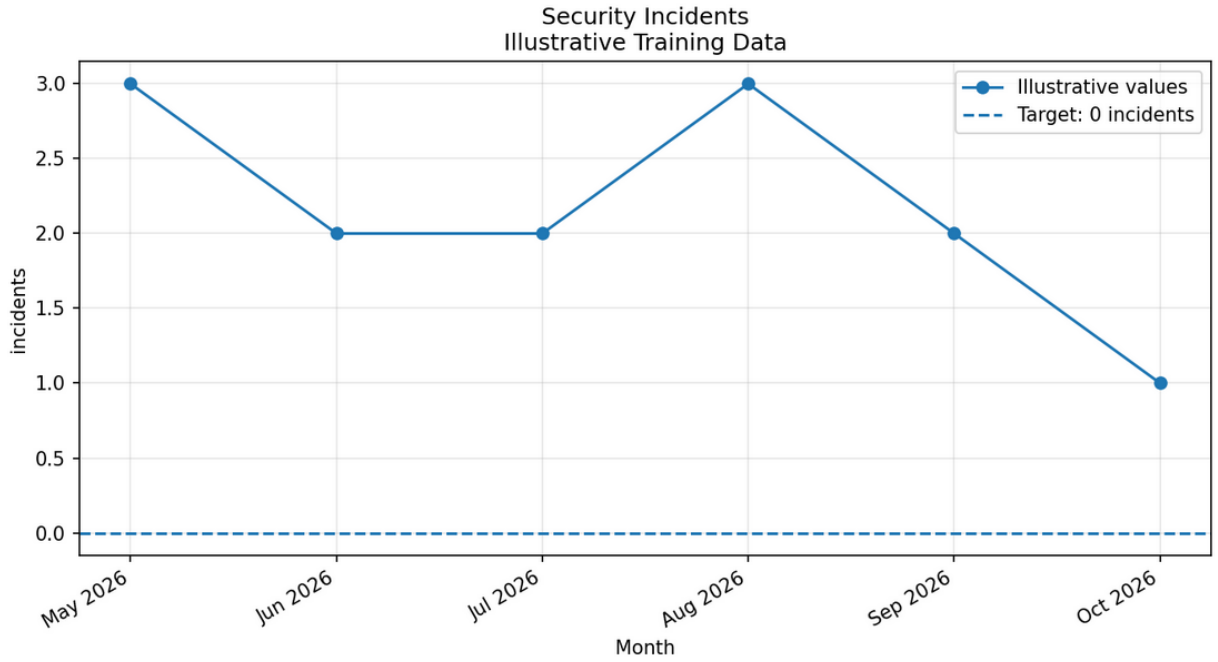


Vulnerability Remediation Time



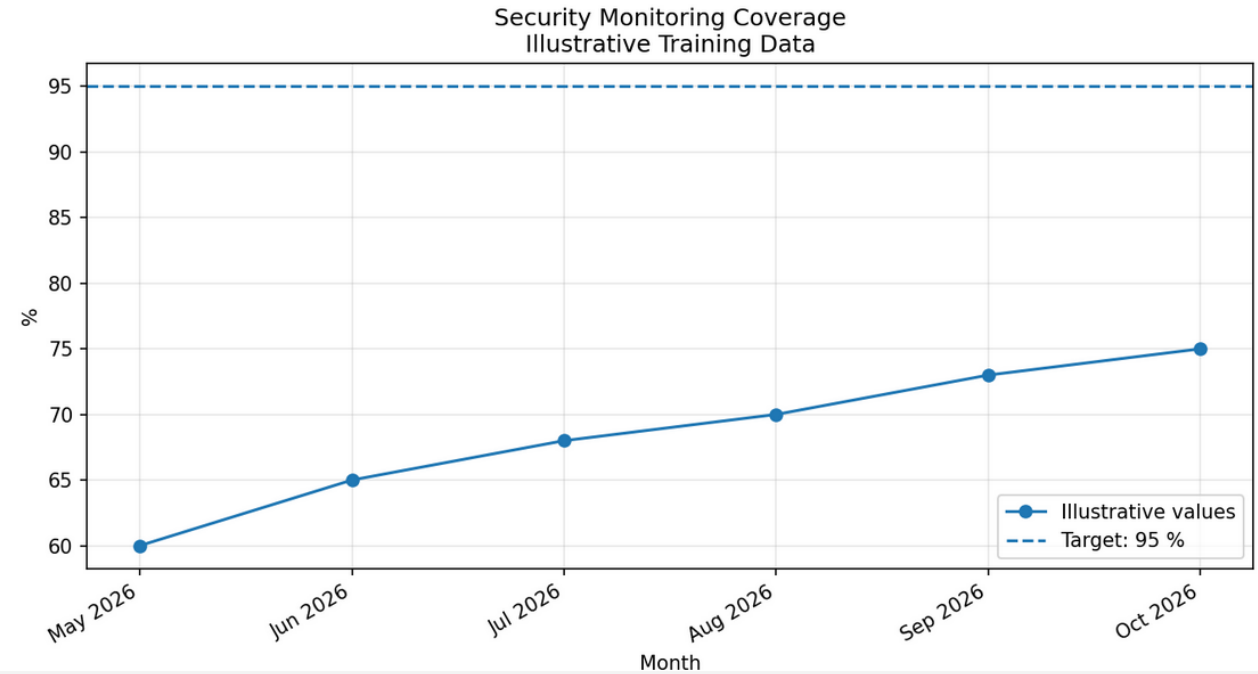
Illustrative trend data, not verified organisational measurements.

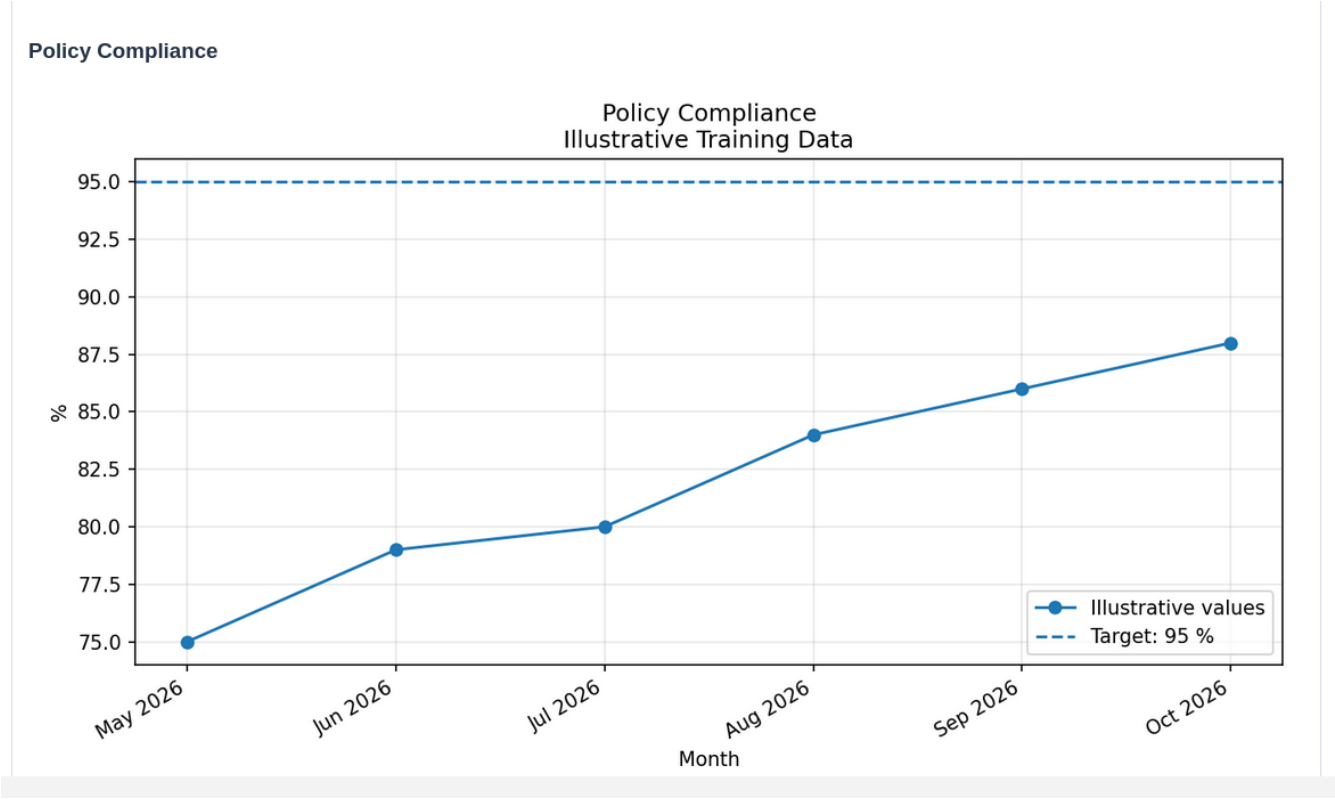
Security Incidents



Illustrative trend data, not verified organisational measurements.

Security Monitoring Coverage





Supporting evidence: executive_dashboard.html, governance_metrics_data.json and the images in metrics_visualizations/.

4.3 Board-Level Security Governance Report

Executive Summary

The security governance simulation identified unresolved technical weaknesses that require management attention. The post-control vulnerability scan continued to report a critical Apache Struts S2-045 finding and a high-severity MySQL default-credentials finding. Independent testing also confirmed that the web container could reach the database on TCP port 3306.

Although the simulated remediation workflow recorded the relevant status flags as complete, the technical retests did not demonstrate that these weaknesses had been resolved. Management should therefore treat the affected controls as unresolved until corrective actions have been implemented and independently verified.

Top Three Security Concerns

1. Critical Apache Struts vulnerability

The post-control scan continued to identify the Struts S2-045 vulnerability. The repeat laboratory check also returned the expected vulnerability marker. The remediation owner should investigate the vulnerable application, implement an effective correction or replacement, and arrange independent retesting.

2. Weak database authentication

The MySQL test successfully authenticated using the configured default root password. The database owner should replace the default credential, review account privileges and confirm that unauthorised access is restricted.

3. Web-to-database connectivity and segmentation

The web container could connect to the database on TCP port 3306. The scanner's separate segmentation check remained inconclusive because of a decoding error. The network owner should validate the intended connectivity requirements, apply least-privilege restrictions and repeat the tests.

Recommended Management Actions

Priority	Recommended action	Responsible owner	Closure evidence
Critical	Correct the Struts vulnerability or replace the vulnerable application component.	Application/Infrastructure Manager	Successful vulnerability retest and documented change
High	Replace default database credentials and review database privileges.	Database Administrator	Authentication test and access-control evidence
High	Restrict web-to-database communication to approved paths.	Network Engineer	Successful permitted/denied connectivity tests
Follow-up	Correct the scanner's segmentation-check error.	Security Engineer	Valid scan result and independent connectivity evidence
Governance	Track owners, deadlines, exceptions and retest results.	Security Governance Lead	Updated governance tracker and review record

Board Oversight and Reporting

Senior management should review the outstanding findings, confirm that accountable owners and remediation deadlines have been assigned, and escalate overdue actions according to the organisation's risk procedures.

A finding should be closed only after appropriate technical evidence demonstrates that the control objective has been achieved. Where remediation is delayed or cannot be completed, the residual risk should be documented and referred to an authorised decision-maker.

Metrics and Reporting Limitations

The generated trend charts use illustrative training data. They are included to demonstrate how governance metrics can be presented and are not actual organisational measurements.

Management decisions should rely on the observed laboratory evidence and subsequent verified remediation results. Simulated patch-status flags and illustrative metrics must not be treated as proof that controls are effective.

Conclusion

The laboratory demonstrates why security governance requires clear ownership, timely remediation, reliable monitoring, management escalation and independent verification. The critical and high-severity findings remain unresolved or unverified, and further technical remediation and retesting are required before the corresponding controls can be reported as effective.